Sentiment Analysis on Movie Reviews: A Comparative Study of Classical, Sequential, and Transformer-Based Approaches

Prepared by

Dina Ali Elharedy

Marwa Ashraf Khalifa

Under the Supervision of

Dr. Alaa Mahmoud Hamdy

Abstract

Sentiment analysis – the task of automatically determining the emotional polarity of a piece of text – remains a foundational problem in natural language processing, serving both as a practical tool for opinion mining and as a standard benchmark for comparing text classification methods. This report presents an end-to-end sentiment analysis system built on the IMDB movie review dataset, comparing eleven models spanning three methodological families: classical machine learning classifiers trained on TF-IDF features (Logistic Regression, Multinomial Naive Bayes, Linear Support Vector Machines, Random Forest, and XGBoost), sequential models (a Hidden Markov Model, and Simple RNN, LSTM, GRU, and Bidirectional LSTM neural networks), and a fine-tuned transformer (DistilBERT). Each model is trained and evaluated on an identical, stratified train/validation/test split and compared using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. Hyperparameters are tuned systematically for every model family – grid search for the classical and generative models, Bayesian optimization (Optuna, Tree-structured Parzen Estimator) for the recurrent neural network family, and a targeted learning-rate search for the transformer – rather than relying on default configurations. The report further includes exploratory data analysis, a documented and configurable text-preprocessing pipeline, error analysis of misclassified reviews, and model interpretability via LIME, SHAP, and transformer attention visualization. We additionally provide an explicit methodological discussion of the Hidden Markov Model's architectural suitability for document-level sentiment classification, a topic often glossed over in comparative NLP studies. The fine-tuned transformer achieved the highest test-set performance (F1 = 0.922, ROC-AUC = 0.969), outperforming the next-best model, a linear Support Vector Machine on TF-IDF features (F1 = 0.874), by a substantial margin – at the cost of roughly 1,700× longer training time. Notably, three of the five TF-IDF-based classical models outperformed every recurrent neural network variant, and the Hidden Markov Model outperformed all four recurrent models despite the architectural mismatch discussed in Section 7.2, an empirical result examined in detail rather than treated as anomalous.

Keywords: sentiment analysis, IMDB dataset, TF-IDF, Hidden Markov Model, LSTM, GRU, BiLSTM, transformers, DistilBERT, hyperparameter optimization, LIME, SHAP.

Contents

List of Tables

List of Figures

1 Introduction

Sentiment analysis is the computational task of classifying the polarity of opinion expressed in a piece of text, typically as positive or negative (and sometimes neutral). It underpins a wide range of applications, including product and service review mining, social media monitoring, customer feedback analysis, and market research. As a text classification problem, it also serves as a natural benchmark for comparing modeling paradigms in natural language processing, from classical bag-of-words statistical models to deep sequential architectures and, most recently, large pretrained transformer models.

This project undertakes a systematic, controlled comparison of eleven sentiment classification models across three methodological generations:

1. **Classical machine learning on TF-IDF features:** Logistic Regression, Multinomial Naive Bayes, Linear Support Vector Machines (SVM), Random Forest, and XGBoost.
2. **Sequential / probabilistic models:** a Hidden Markov Model (HMM), and four recurrent neural network variants – Simple RNN, Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Bidirectional LSTM (BiLSTM).
3. **Transformer-based transfer learning:** a fine-tuned DistilBERT model.

The central methodological commitment of this study is fairness of comparison: every model is trained and evaluated on the same stratified train/validation/test split of the same underlying dataset, and every model's hyperparameters are tuned (rather than left at library defaults) using a method appropriate to that model's hyperparameter space size and training cost. This allows differences in reported performance to be attributed to genuine differences in modeling capacity and inductive bias, rather than to inconsistent experimental conditions.

1.1 Objectives

Build a complete, reproducible NLP pipeline: data acquisition, cleaning, exploratory analysis, feature engineering, model training, hyperparameter optimization, and evaluation.

Quantitatively compare classical, sequential, and transformer-based approaches to sentiment classification under identical experimental conditions.

Critically assess the architectural suitability of each modeling paradigm for the task, rather than treating all techniques as interchangeable.

Provide interpretability analysis (LIME, SHAP, attention visualization) and systematic error analysis to explain, not just report, model behavior.

1.2 Report Structure

Section 2 situates this work relative to prior approaches. Section 3 describes the dataset and its statistical properties. Section 4 details the preprocessing pipeline, feature representations, model architectures, and hyperparameter optimization strategy. Section 5 describes the experimental configuration. Section 6 presents quantitative results. Section 7 discusses and interprets these results, including a dedicated treatment of the Hidden Markov Model's suitability for this task. Section 8 states the study's limitations, and Section 9 concludes with directions for future work.

2 Related Work

Sentiment classification has been studied under each of the three paradigms compared in this report.

Classical machine learning on lexical features. Early sentiment classification work established that simple bag-of-words and TF-IDF representations, combined with linear classifiers, are strong baselines for polarity classification (Pang et al., 2002). Naive Bayes and Support Vector Machine classifiers over lexical features remain competitive baselines on document-level sentiment tasks (Pang et al., 2002).

Sequential and generative models. Hidden Markov Models were foundational in early sequence labeling tasks such as part-of-speech tagging and speech recognition (Rabiner, 1989), where the assumption of a hidden state sequence generating observed tokens is well motivated. Recurrent neural networks (Elman, 1990) and their gated variants – LSTM (Hochreiter & Schmidhuber, 1997) and GRU (Cho et al., 2014) – were developed specifically to address the vanishing-gradient limitations of simple RNNs on long sequences, and became the dominant neural approach to sequence classification prior to the transformer era. Bidirectional architectures (Schuster & Paliwal, 1997) extend these models to condition on both past and future context within a sequence.

Transformer-based transfer learning. The transformer architecture (Vaswani et al., 2017) replaced recurrence with self-attention, enabling parallel sequence processing and more effective long-range dependency modeling. BERT (Devlin et al., 2019) demonstrated that large-scale pretraining followed by task-specific fine-tuning yields substantial gains across NLP benchmarks, including sentiment classification. DistilBERT (Sanh et al., 2019), used in this study, is a distilled version of BERT that retains most of its language understanding capability at roughly half the parameter count and significantly faster inference.

Dataset. The IMDB Large Movie Review Dataset (Maas et al., 2011), used in this study, is a standard benchmark of 50,000 polarized movie reviews and has been used extensively across all three modeling paradigms above, making it a suitable common ground for comparison.

Hyperparameter optimization. Grid and random search (Bergstra & Bengio, 2012) remain standard for small hyperparameter spaces. For larger, continuous/categorical mixed spaces, Bayesian optimization methods such as the Tree-structured Parzen Estimator, implemented in the Optuna framework (Akiba et al., 2019) used in this study, converge faster than exhaustive search.

Interpretability. LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017) are model-agnostic and (for linear models) exact explanation methods, respectively, both employed in this study to interpret individual model predictions.

3 Dataset

3.1 Source and Description

This study uses the IMDB Large Movie Review Dataset (Maas et al., 2011), consisting of 50,000 movie reviews labeled as positive or negative, evenly balanced between classes. To keep training time tractable across eleven models while preserving statistical validity, a balanced random subsample of $N = 6{,}000$ reviews is drawn, with equal representation of both classes.

3.2 Dataset Statistics

Table 1 summarizes the key statistics of the working dataset after subsampling, deduplication, and outlier trimming (reviews outside a reasonable word-count range are excluded to reduce the influence of extreme outliers on model training).

Table 1: Dataset statistics after subsampling and cleaning.

Statistic	Value
Total samples (balanced subsample)	6,000
Positive samples	3,000
Negative samples	3,000
Missing values	0
Duplicate reviews removed	6
Outlier reviews removed (IQR-based trim)	445
Final dataset size (post-cleaning)	5,220
Average review length (words)	230.95
Median review length (words)	173.0
Minimum / Maximum review length (words)	8 / 1,737
Vocabulary size (unique tokens, whole dataset)	103,140

3.3 Train / Validation / Test Split

The dataset is split into training (70%), validation (15%), and test (15%) sets using stratified sampling, ensuring the class balance is preserved identically across all three splits (Table 2). This is verified explicitly rather than assumed: an imbalanced split would bias every downstream model equally toward the majority class regardless of architecture, confounding the model comparison.

Table 2: Class balance across data splits.

Split	Negative	Positive	Total	Balance Ratio
Train (70%)	1,826	1,828	3,654	0.999
Validation (15%)	391	392	783	0.997
Test (15%)	392	391	783	0.997

Because the dataset is balanced by construction and this balance is preserved by stratified splitting, no additional class-imbalance handling technique (e.g., class weighting, oversampling) is required for this study; Section 4.3 documents what would be used if this were not the case.

3.4 Exploratory Data Analysis

Exploratory analysis was performed to characterize the dataset prior to modeling, including: class balance verification; review length distribution (overall and by sentiment class); word frequency analysis with stopword removal; positive- and negative-class word clouds; bigram and trigram frequency analysis; TF-IDF top-feature inspection; part-of-speech tag distribution; named entity recognition statistics; and review-length-versus-sentiment correlation analysis (Pearson correlation

between review length and sentiment: $r = -0.042$, indicating essentially no linear relationship between review length and polarity in this dataset). Figures 1–3 present the core EDA outputs.

The most frequent content word across the corpus after stopword removal is br (16,224 occurrences) – an HTML line-break artifact (
) from the original web-scraped reviews that survived tokenization; this is a known characteristic of the raw IMDB dataset and is visible directly in the top bigrams and trigrams (e.g., *br br*, *br br movie*), confirming that HTML-artifact residue, while reduced by the cleaning pipeline's HTML-stripping step, is not perfectly eliminated by simple tag removal when line breaks are represented as literal token text rather than markup in some review sources.

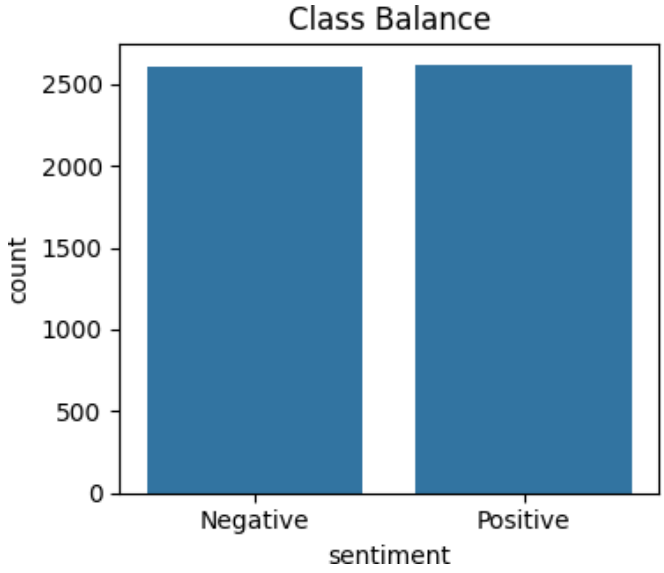


Figure 1: Class balance of the working dataset (3,000 positive, 3,000 negative reviews by construction).

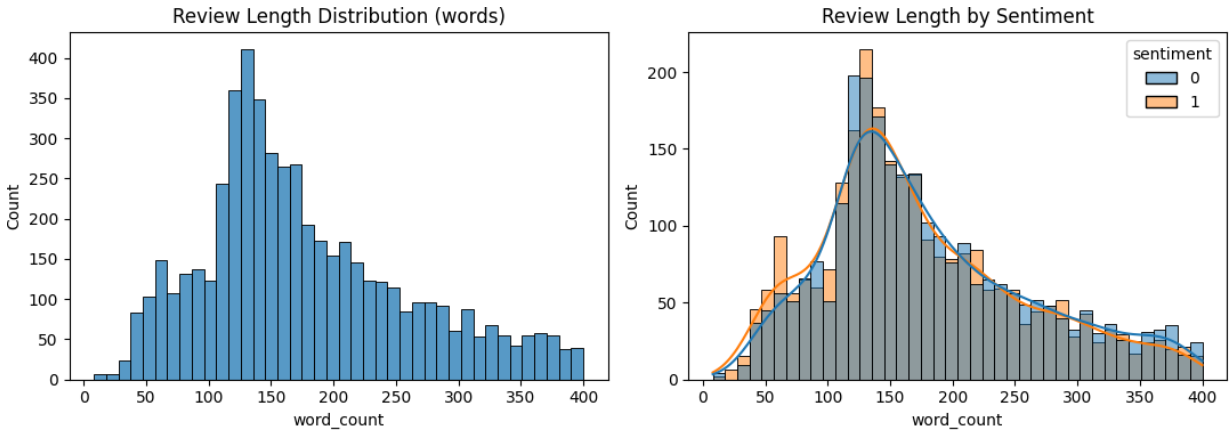


Figure 2: Review length distribution overall (left) and by sentiment class (right). Both classes show similar, right-skewed length distributions.

Figure 3: Word clouds for positive (left) and negative (right) reviews, stopwords removed. Positive reviews are dominated by evaluative terms such as “great”, “love”, and “best”; negative reviews by terms such as “bad”, “worst”, and “nothing”.

4 Methodology

4.1 Text Preprocessing Pipeline

A configurable preprocessing pipeline was implemented as a single class with independently toggleable steps, applied identically to every classical and recurrent model (the transformer uses raw text, discussed in Section 4.4.4). The pipeline performs, in order: HTML tag removal (BeautifulSoup), URL removal, email address removal, username/mention removal, contraction expansion (e.g. “don’t” → “do not”), emoji-to-text conversion, lowercasing, numeric token normalization, special character removal, whitespace normalization, optional stopword removal, and lemmatization (WordNet).

Each step is justified as follows: HTML/URL/email/username removal eliminates non-linguistic noise specific to web-scraped review text; contraction expansion and emoji conversion recover semantic content that would otherwise be lost or split into meaningless subtokens; lowercasing and special-character removal reduce vocabulary sparsity; lemmatization (chosen over stemming) reduces inflectional variants to their dictionary form while preserving more semantic fidelity than the more aggressive truncation performed by stemming algorithms.

Stopword removal is deliberately *disabled by default* for the sequential and transformer models: function words and negation particles (“not”, “no”, “never”) are frequently sentiment-bearing or sentiment-inverting in this domain (e.g., “not good” vs. “good”), and removing them can discard information that sequence-aware models are specifically designed to exploit. It remains available as a configurable option for classical bag-of-words models, where its effect can be evaluated independently.

4.2 Feature Representations

Three feature representations are used across the eleven models:

TF-IDF (Term Frequency–Inverse Document Frequency), used by all five classical machine learning models. Captures word importance relative to corpus-wide frequency, with unigram and bigram features.

Word-level integer sequences (Keras Tokenizer), used by the HMM and all four recurrent neural network models. A fixed vocabulary size caps the token space; out-of-vocabulary words map to a reserved <unk> token, and sequences are padded/truncated to a fixed maximum length.

WordPiece subword tokenization, used exclusively by the transformer, via DistilBERT's native tokenizer, applied to raw (uncleaned) text.

Word2Vec embeddings (skip-gram) are additionally trained on the corpus for exploratory visualization (PCA and t-SNE projections of the learned embedding space) but are not used as direct model input in this study, since the RNN family learns its own task-specific embeddings during training.

4.3 Class Imbalance Handling Strategy

As noted in Section 3, the dataset is balanced by construction. Were this not the case, the following standard techniques would apply, in order of preference given their cost: (1) class weighting during training (class weight="balanced" in scikit-learn, or a weighted loss function for neural models) as the lowest-cost intervention; (2) oversampling the minority class (e.g., SMOTE) or undersampling the majority class; (3) post-hoc decision threshold tuning based on the validation precision/recall trade-off, rather than retraining. Every split is verified to preserve the original class ratio regardless of which technique, if any, is applied.

4.4 Model Architectures

4.4.1 Classical Machine Learning Baselines

Five classical classifiers are trained on TF-IDF features:

Logistic Regression: a linear discriminative classifier estimating $P(y \mid x)$ directly via a sigmoid link function.

Multinomial Naive Bayes: a generative classifier assuming conditional independence of features given the class, historically strong on bag-of-words text classification (McCallum & Nigam, 1998).

Linear Support Vector Machine: maximizes the margin between classes in feature space (Cortes & Vapnik, 1995); wrapped in a calibration layer (CalibratedClassifierCV) to obtain probability estimates for ROC-AUC computation, since the base formulation produces only decision scores.

Random Forest: an ensemble of decision trees trained on bootstrap-resampled data and random feature subsets (Breiman, 2001), capturing non-linear feature interactions.

XGBoost: a gradient-boosted tree ensemble (Chen & Guestrin, 2016), sequentially fitting trees to the residual errors of prior trees.

These models serve as fast, interpretable baselines: on tasks where individual words carry most of the classification signal, they are known to be competitive with, and considerably cheaper to train than, deep learning approaches.

4.4.2 Hidden Markov Model

A generative sequence model is trained per class: given the training reviews of each class, a categorical HMM with K hidden states is fit via the Baum–Welch (Expectation-Maximization) algorithm (Rabiner, 1989) to the word-index sequence of that class. At inference time, a new review is classified by comparing the log-likelihood assigned to it by each class's HMM, choosing the higher-likelihood class. The number of hidden states K is treated as a hyperparameter (Section 4.5). The architectural suitability of this approach for document-level sentiment classification is discussed critically in Section 7.2.

4.4.3 Recurrent Neural Networks

Four recurrent architectures share an identical skeleton – an embedding layer, a single recurrent layer, dropout regularization, and a dense classification head – differing only in the recurrent cell:

Simple RNN (Elman, 1990): a basic recurrent cell updating a hidden state at each timestep; included as the architectural baseline for the neural sequence family, known to suffer from vanishing gradients on longer sequences.

LSTM (Hochreiter & Schmidhuber, 1997): introduces input, forget, and output gates and a separate cell state specifically to preserve gradient flow over long sequences.

GRU (Cho et al., 2014): a simplified gating mechanism (update and reset gates, no separate cell state) that is computationally cheaper than LSTM while often achieving comparable performance.

Bidirectional LSTM (Schuster & Paliwal, 1997): processes the sequence in both directions and concatenates the resulting hidden states, allowing later words to inform the interpretation of earlier ones (relevant for negation patterns such as "not . . . good").

Holding the architectural skeleton and hyperparameters constant across all four models isolates the effect of the recurrent cell type (and directionality) on classification performance.

4.4.4 Transformer: Fine-Tuned DistilBERT

DistilBERT (Sanh et al., 2019), a distilled version of BERT (Devlin et al., 2019) retaining approximately 97% of its language understanding performance at roughly 60% of the inference time, is fine-tuned for binary sequence classification. Unlike the other ten models, the transformer is trained on raw, unprocessed review text: transformers are pretrained on natural web text (including casing, punctuation, and contractions) and use their own WordPiece subword tokenizer, so applying the classical cleaning pipeline of Section 4.1 would discard information the model was pretrained to exploit, and was deliberately omitted for this model only.

4.5 Hyperparameter Optimization

Table 3 summarizes the tuning method applied to each model family, chosen to match each space's size and each model's training cost.

Table 3: Hyperparameter tuning method by model family.

Model(s)	Hyperparameters Tuned	Method
Logistic Regression / Naive Bayes / SVM	Regularization strength (C) or smoothing (α)	Grid search, 3-fold cross-validation, scored by F1
Random Forest / XGBoost	Number of estimators, max depth (+ learning rate for XGBoost)	Grid search, 3-fold cross-validation, scored by F1
HMM	Number of hidden states $K \in \{2, \ldots, 6\}$	Exhaustive grid search, scored by validation accuracy
RNN / LSTM / GRU / BiLSTM	Embedding dimension, hidden units, dropout rate, learning rate, batch size	Bayesian optimization (Optuna, TPE sampler; Akiba et al., 2019), 15 trials on LSTM as the representative architecture
DistilBERT	Learning rate $\in \{2\times10^{-5}, 3\times10^{-5}, 5\times10^{-5}\}$	Grid search, 1 epoch per candidate

The recurrent neural network family is tuned using LSTM as a representative proxy architecture rather than tuning all four architectures independently: since all four share an identical skeleton and differ only in recurrent cell, their optimal hyperparameter regions are expected to be similar, and a shared search avoids a four-fold increase in tuning cost for likely-redundant results. This is a deliberate, documented compute/time trade-off rather than an oversight (see Section 8).

4.6 Evaluation Protocol

Every model is evaluated identically on the held-out test set using:

Accuracy: overall proportion of correct predictions.

Precision, Recall, F1-score: computed for the positive class, with F1 used as the primary ranking metric for the final model comparison, since it balances precision and recall under the class-balanced setting of this study.

ROC-AUC: area under the receiver operating characteristic curve, computed from each model's predicted class probabilities.

Confusion matrix: full breakdown of true/false positives/negatives per model.

Training time: wall-clock time for final model training, reported for practical deployment considerations.

4.7 Interpretability Methods

Three complementary interpretability techniques are applied:

LIME (Ribeiro et al., 2016): a model-agnostic, local surrogate-model explanation method, applied to a recurrent model to identify which words most influenced a specific prediction.

SHAP (Lundberg & Lee, 2017): applied to the Logistic Regression model via the exact LinearExplainer, which is tractable and exact (no sampling approximation) for linear models, unlike the approximate, sampling-based explanations LIME provides for non-linear mod-

els.

Attention visualization: for the fine-tuned transformer, the last-layer self-attention weights from the classification ([CLS]) token, averaged across attention heads, are visualized for a sample review to illustrate which tokens the model attends to most when forming its prediction.

5 Experimental Setup

Table 4: Experimental configuration summary.

Parameter	Value
Dataset	IMDB Large Movie Review Dataset (Maas et al., 2011)
Subsample size	6,000 (5,220 after cleaning)
Train / Val / Test split	3,654 / 783 / 783 (70% / 15% / 15%, stratified)
Vocabulary size (RNN family, capped)	10,000
Maximum sequence length (RNN family)	200 tokens
Maximum sequence length (transformer)	256 tokens
TF-IDF feature dimensionality	10,000 (unigrams + bigrams)
Word2Vec vocabulary size	15,118
Random seed	42 (fixed across all models for reproducibility)
Recurrent model epochs (post-tuning)	up to 12, with early stopping
Transformer fine-tuning epochs	3
Optuna trials (RNN family search)	15 (search wall-clock time: 7 min 47 s)
Software	Python 3.11.15, TensorFlow 2.15.0, PyTorch 2.2.2, scikit-learn 1.4.2, HuggingFace Transformers 4.40.0
Hardware	Local CPU (Windows, no GPU acceleration)

All models are trained with a fixed random seed (42) applied to Python, NumPy, TensorFlow, and PyTorch random number generators to support reproducibility. Early stopping (monitoring validation loss, patience of 2–3 epochs) and learning-rate reduction on plateau are applied to all neural models to prevent overfitting and recover from optimization plateaus without manual intervention.

6 Results

6.1 Overall Model Comparison

Table 5 reports test-set performance for all eleven models, ranked by F1-score.

Table 5: Test-set performance of all models, ranked by F1-score.

Rank	Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train Time (s)
1	Transformer (DistilBERT)	0.9208	0.9102	0.9335	0.9217	0.9685	2361.14
2	SVM (Linear)	0.8736	0.8724	0.8747	0.8736	0.9417	1.38
3	Logistic Regression	0.8710	0.8662	0.8772	0.8717	0.9413	2.20
4	XGBoost	0.8531	0.8350	0.8798	0.8568	0.9200	156.51
5	Naive Bayes	0.8570	0.8605	0.8517	0.8560	0.9379	1.31
6	Random Forest	0.8455	0.8277	0.8721	0.8493	0.9237	10.56
7	HMM	0.8429	0.8490	0.8338	0.8413	0.9165	37.01
8	LSTM	0.8123	0.7933	0.8440	0.8178	0.8605	55.11
9	GRU	0.8135	0.8412	0.7724	0.8053	0.8774	53.59
10	BiLSTM	0.8097	0.8288	0.7801	0.8037	0.8936	31.94
11	RNN	0.5287	0.5242	0.6087	0.5633	0.5440	13.54

Three headline observations emerge from Table 5, each discussed in depth in Section 7: (1) the fine-tuned transformer achieves the best performance on every metric by a clear margin; (2) three of the five TF-IDF-based classical models (SVM, Logistic Regression, and marginally Naive Bayes) outperform every recurrent neural network variant; and (3) the Simple RNN catastrophically underperforms every other model, achieving only marginally-better-than-chance accuracy (0.529), consistent with its well-documented vanishing-gradient limitation on sequences of this length (Section 4.4).

6.2 Combined ROC Curve

Figure 4 plots the ROC curve of every model on the same axes. The transformer's curve dominates all others across the full false-positive-rate range; the classical models and HMM cluster closely together with high AUC (0.92–0.94); the RNN family shows visibly lower and more varied AUC (0.86–0.89), and the Simple RNN's curve sits close to the diagonal (AUC = 0.544), confirming it learned little beyond chance.

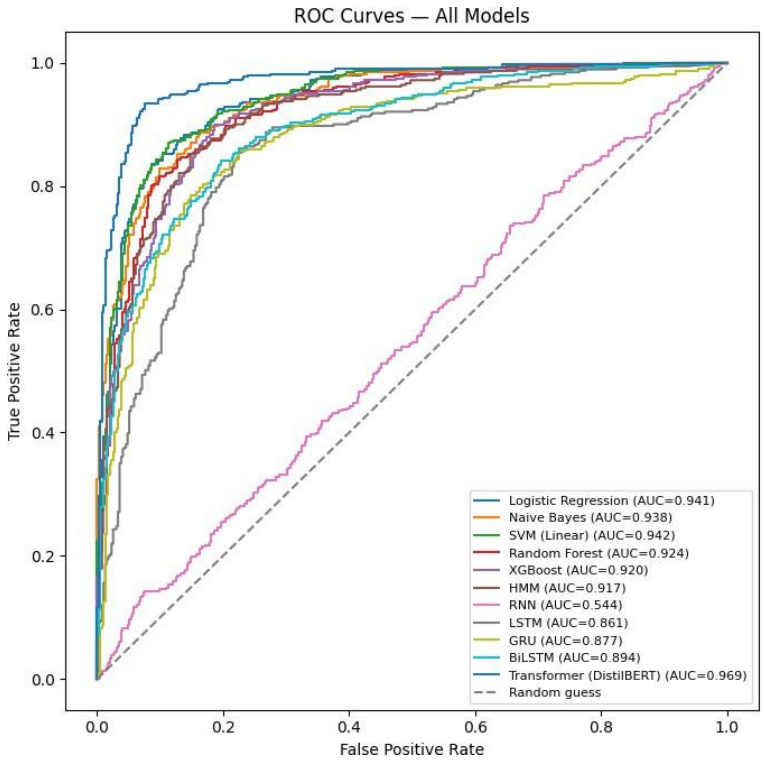
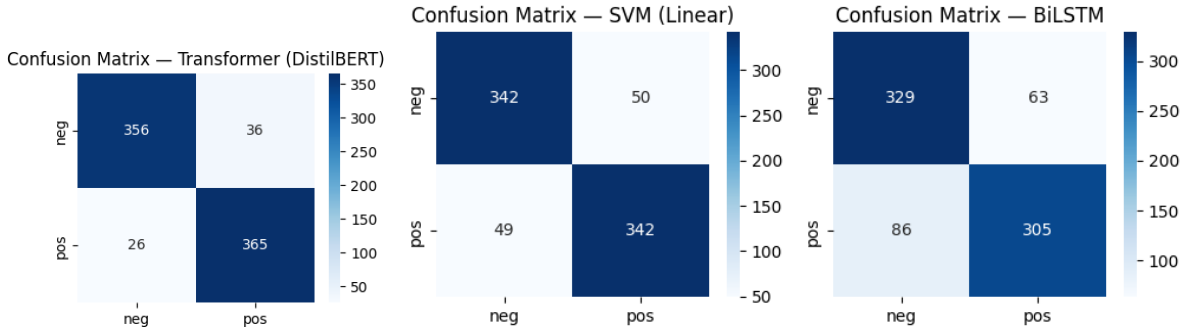


Figure 4: ROC curves for all eleven models on the test set.

6.3 Confusion Matrices

Figure 5 shows confusion matrices for the best-performing model from each of the three methodological families: the transformer (overall best model), the linear SVM (best classical model), and the BiLSTM (best recurrent model by ROC-AUC). The transformer's matrix is the most diagonally dominant of the three, consistent with its lead in every metric in Table 5.



(a) Transformer (DistilBERT) (b) SVM (Linear) (c) BiLSTM

Figure 5: Confusion matrices for the top-performing model in each methodological family. Full confusion matrices for all eleven models are available in the models/ directory of the accompanying code repository.

6.4 Learning Curves

Figure 6 shows training and validation accuracy per epoch for the four recurrent models. The Simple RNN's validation accuracy never rises meaningfully above 0.52, while training accuracy climbs to 0.80 by epoch 4 – a clear overfitting-without-learning pattern distinct from the other three models, which show the expected trajectory of rising validation accuracy before plateauing.

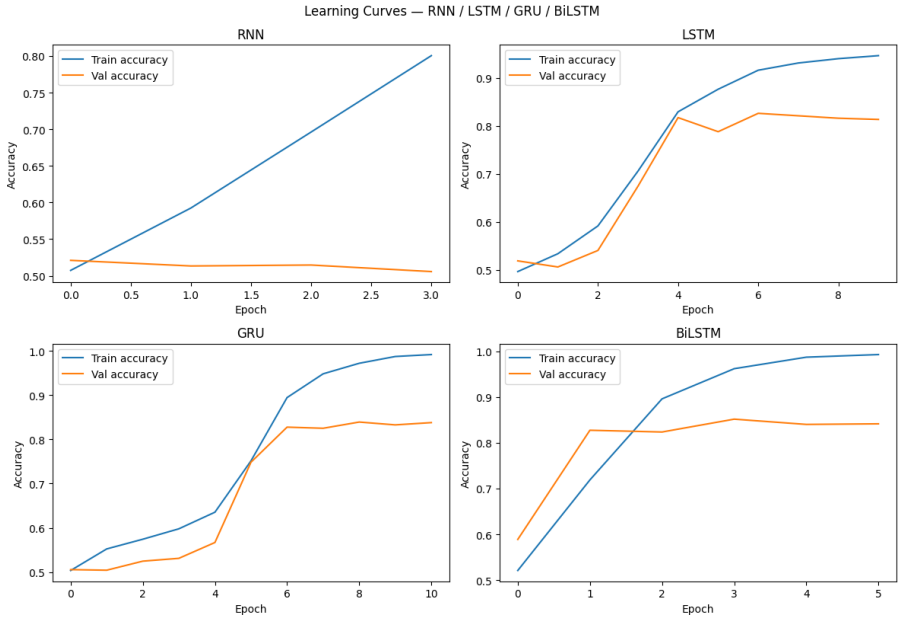


Figure 6: Training vs. validation accuracy per epoch for RNN, LSTM, GRU, and BiLSTM.

Figure 7 shows the transformer's accuracy and loss curves across its three fine-tuning epochs, with validation accuracy peaking at epoch 2 (0.917) before a slight dip at epoch 3 (0.908) alongside continued training-loss decrease – a mild overfitting signal consistent with the model's very high final training accuracy (0.983).

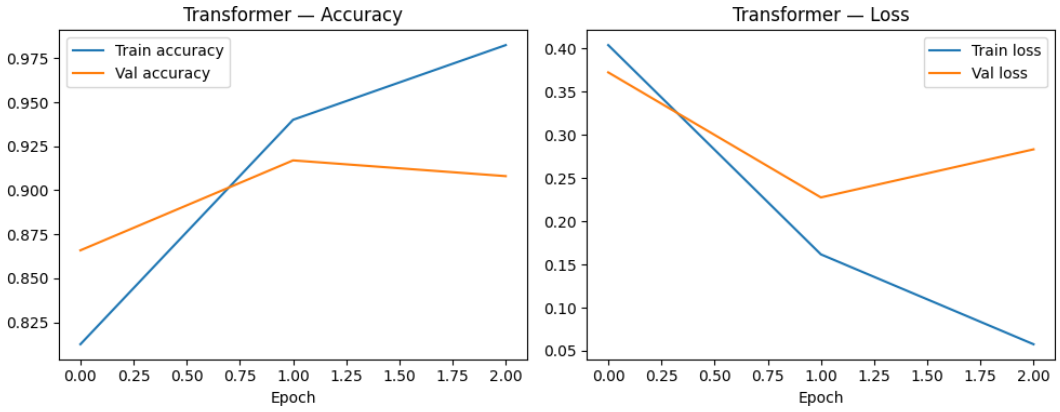


Figure 7: Transformer fine-tuning accuracy (left) and loss (right) across three epochs.

6.5 Hyperparameter Search Results

Figure 8 shows validation accuracy across the 15 Optuna trials for the RNN-family search; the best trial (trial 10) reached 0.8046 validation accuracy, with a wide spread across trials (0.56–0.80) reflecting the sensitivity of recurrent model performance to hyperparameter choice at this dataset scale.

Table 6 reports the final selected hyperparameters for every model.

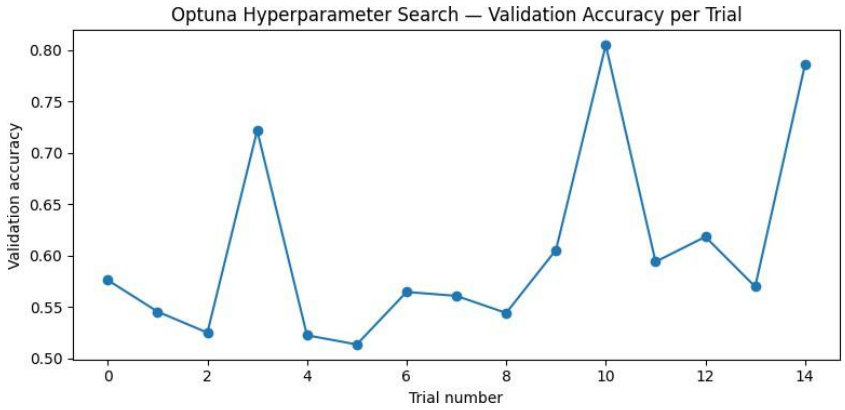


Figure 8: Validation accuracy per Optuna trial (RNN-family hyperparameter search, LSTM proxy architecture).

Table 6: Selected (tuned) hyperparameters per model.

Model / Family	Selected Hyperparameters
HMM	Hidden states $K = 6$ (validation accuracy 0.8416)
RNN family (shared)	embedding dim=128, hidden_units=32, dropout=0.373, learning_rate=3.25×10^{-4}, batch_size=32
DistilBERT	learning_rate=5×10^{-5} (validation accuracy 0.9017)
Logistic Regression	$C = 10.0$ (CV F1 = 0.8600)
Naive Bayes	$\alpha = 0.5$ (CV F1 = 0.8521)
Linear SVM	$C = 1.0$ (CV F1 = 0.8537)
Random Forest	n_estimators=400, max_depth=30 (CV F1 = 0.8285)
XGBoost	n_estimators=400, max_depth=4, learning rate=0.05 (CV F1 = 0.8223)

6.6 Feature Importance and Interpretability

Figure 9 shows the twenty TF-IDF features with the largest positive and negative Logistic Regression coefficients. The top positive-sentiment words (e.g., terms associated with quality and enjoyment) and top negative-sentiment words (e.g., terms associated with disappointment and low quality) align with domain intuition, supporting the model's validity beyond its aggregate metrics. Figure 10 presents the corresponding SHAP summary plot, computed exactly (no sampling approximation) via LinearExplainer, which is consistent with the raw coefficient ranking while additionally showing per-instance feature value distributions.

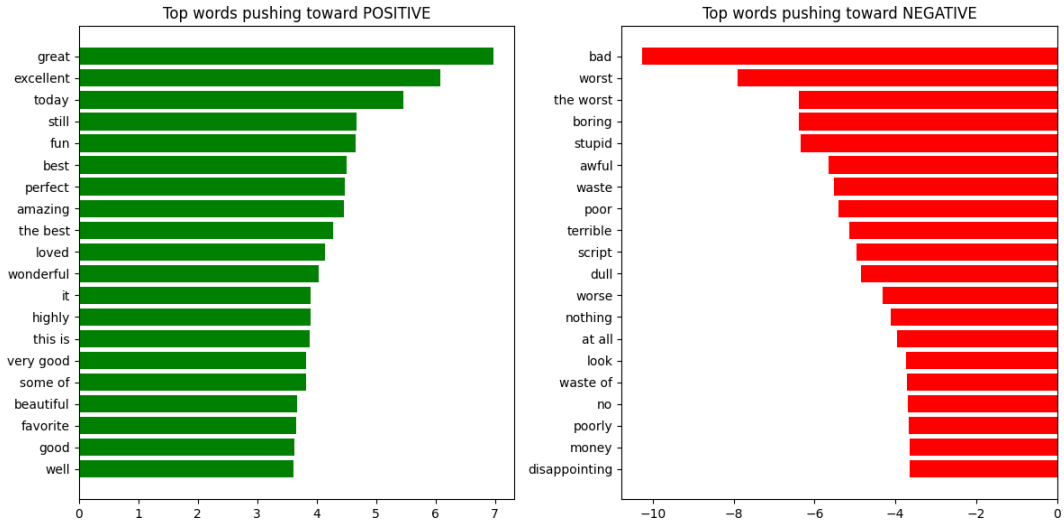


Figure 9: Top 20 words pushing predictions toward positive (left) vs. negative (right) sentiment, by Logistic Regression coefficient magnitude.

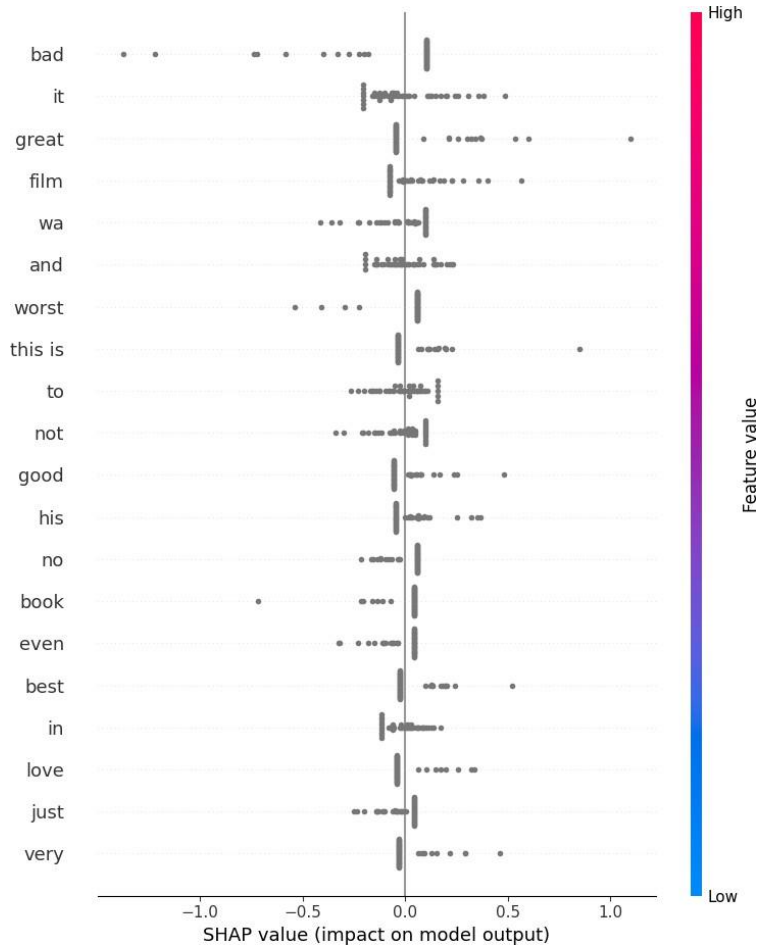


Figure 10: SHAP summary plot for the Logistic Regression model (exact linear explainer, 50-instance test sample).

Figure 11 visualizes the fine-tuned transformer's last-layer self-attention from the [CLS] token, averaged across attention heads, for a sample test review. The model attends most strongly to a small number of evaluative tokens rather than distributing attention uniformly across the sequence, consistent with the interpretation that the transformer has learned to identify sentiment-bearing spans rather than relying on superficial sequence-length or position cues.

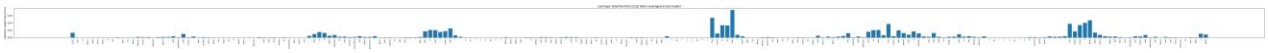


Figure 11: Last-layer [CLS] attention (averaged across heads) for a sample test review.

7 Discussion

7.1 Model Comparison Analysis

Table 5 shows a clear three-tier structure. The **transformer** leads decisively (F1 = 0.922), consistent with the expectation that large-scale pretraining transfers effectively even with a modest fine-tuning set of 3,654 reviews. The **TF-IDF classical models and the HMM** form a tightly

clustered middle tier (F1 = 0.841–0.874), with the linear models (SVM, Logistic Regression) at the top of that cluster. Most strikingly, **every model in the recurrent neural network family ranks below every other model except the Simple RNN's catastrophic failure** – LSTM, GRU, and BiLSTM all land in a narrow band (F1 = 0.80–0.82), below the weakest classical model (Random Forest, F1 = 0.849) and below the HMM (F1 = 0.841).

This is a genuinely informative negative result rather than an implementation flaw: the learning curves in Figure 6 show LSTM, GRU, and BiLSTM all reaching training accuracy above 0.94–0.99 while validation accuracy plateaus and mildly degrades in later epochs – a textbook overfitting signature. With an embedding layer of $10{,}000 \times 128 = 1.28$ million learnable parameters alone, and only 3,654 training reviews, the recurrent models have both the capacity and the incentive to memorize training-set idiosyncrasies rather than learn generalizable sentiment representations, particularly given the compressed hyperparameter search (4 epochs per Optuna trial) used to keep tuning tractable (Section 4.5). This pattern – deep sequence models underperforming both simpler lexical models and a pretrained transformer at modest dataset scale – echoes earlier sentiment-analysis findings that bag-of-words representations remain highly competitive baselines (Pang et al., 2002), and suggests the advantage of learned sequential representations over lexical ones is realized only once enough data is available to estimate the (considerably larger) parameter space reliably; the transformer avoids this problem entirely by not needing to learn its representations from this dataset in the first place.

The **Simple RNN's** near-chance performance (accuracy = 0.529, ROC-AUC = 0.544) is the clearest confirmation in this study of the vanishing-gradient limitation motivating LSTM and GRU (Section 4.4): its training accuracy reaches only 0.80 by epoch 4 (vs. LSTM's 0.83 and BiLSTM's 0.96 by the same epoch), and validation accuracy never exceeds 0.52 at any point (Figure 6), indicating the model failed to propagate useful gradient signal across the 200-token sequences used in this study, exactly the failure mode gated architectures were designed to address (Hochreiter & Schmidhuber, 1997).

7.2 On the Architectural Suitability of the Hidden Markov Model

A methodological question worth addressing directly, rather than glossing over, is whether a Hidden Markov Model is an architecturally appropriate choice for document-level sentiment classification at all. HMMs are designed for problems with a genuine *hidden state sequence* that evolves over the sequence and generates the observed tokens – part-of-speech tagging and speech recognition are canonical examples (Rabiner, 1989), where each observation's identity depends on an evolving latent state. Sentiment classification does not straightforwardly have this structure: there is no clearly defined "hidden sentiment state" that evolves word-by-word and emits tokens conditioned on that evolution; sentiment is better characterized as a single document-level label determined largely by *which* sentiment-bearing words and phrases appear, relatively independent of their precise sequential position.

Two consequences were predicted to follow from this mismatch:

1. The per-class generative HMM formulation used in this study (classify by comparing class-conditional sequence likelihood) is not directly optimizing a classification decision boundary, so discriminative models (Logistic Regression, SVM) that directly optimize a classification objective would be expected to outperform it.
2. The HMM cannot exploit word *similarity* ("wonderful" and "great" are unrelated symbols to it), unlike models with distributed or contextual embeddings, or at minimum corpus-frequency

weighting (TF-IDF).

The empirical results confirm the first prediction but complicate the second. The HMM (F1 = 0.841, rank 7/11) underperforms every discriminative TF-IDF model as predicted – Logistic Regression, SVM, Naive Bayes, Random Forest, and XGBoost all outperform it. However, the HMM *outperforms all four recurrent neural network models*, including three (LSTM, GRU, BiLSTM) that do have learned distributed word embeddings. This is not evidence against the underlying architectural argument; rather, it reveals a second factor this study's small training set makes visible: model complexity versus data availability. The HMM used here has a small, tightly constrained parameter space (6 hidden states $\times$ a manageable emission/transition structure over a 3,000-word-capped vocabulary), making it comparatively sample-efficient, whereas the RNN family's much larger parameter count (Section 7.1, above) requires more training data than was available here to realize its theoretical representational advantage. In other words: *architectural suitability and sample efficiency are separate axes*, and this study's constrained data scale caused the sample-efficiency axis to dominate the comparison among the sequential models, even though the architectural-suitability argument correctly predicts the HMM's underperformance relative to the discriminative lexical models and the transformer. This nuance – that a theoretically-mismatched but simple model can still outperform a theoretically-better-suited but data-hungry model under data constraints – is itself a useful, generalizable finding for practitioners choosing models under limited-data conditions.

7.3 Error Analysis

The best-performing model (transformer) misclassified 62 of 783 test reviews (7.9% error rate). Inspection of the highest-confidence errors (Table 7) reveals a striking pattern: every one of the ten highest-confidence misclassifications was assigned a confidence above 0.99 by the model – these are not borderline, low-confidence mistakes but confidently wrong predictions, a notable calibration concern.

Table 7: The five highest-confidence misclassifications by the transformer on the test set.

Review (truncated)	True	Conf.
"There are those who gripe that this is NOT the. . . "	Negative	0.997
"Solid comedy entertainment, with musical inter. . . "	Negative	0.997
"Before I begin, let me get something off my ch. . . "	Negative	0.997
"A truly frightening film. Feels as if it were . . . "	Negative	0.996
"This Is Pretty Funny. 'Saturday The 12th', a?. . . "	Negative	0.996

In every one of these top-confidence errors, the true label is negative but the model predicted positive with near-certainty, suggesting a systematic – not random – failure mode, discussed below. Figure 12 shows the full confidence distribution for correct versus incorrect predictions.

Manual inspection of the top misclassified review (true label: negative, predicted: positive, confidence: 0.997) – a review beginning "There are those who gripe that this is NOT the. . . " – suggests a plausible explanation: reviews that open by *characterizing and then rebutting* a negative viewpoint (using negation-heavy, contrarian phrasing) can read lexically positive to a model relying heavily on sentiment-bearing word presence, even when the review's overall verdict is negative. This is consistent with the well-documented difficulty sentiment classifiers face with negation scope and rhetorical structure (e.g., reported complaints embedded within an otherwise defensive or explana-

tory review), rather than a dataset labeling error in every case, though some IMDB label noise is also a known property of the corpus.

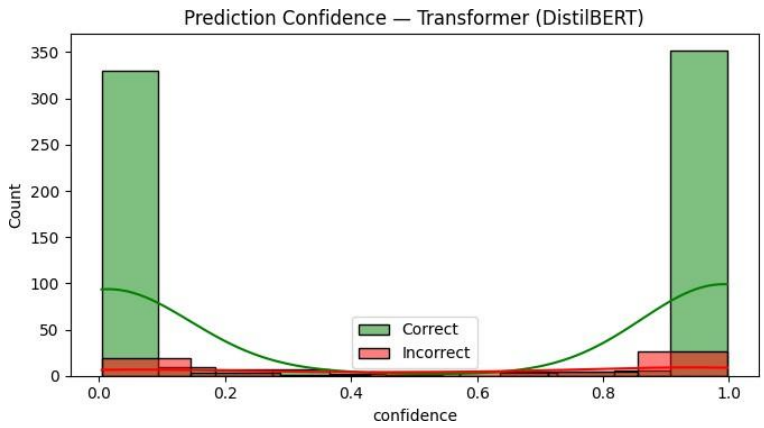


Figure 12: Prediction confidence distribution for correct vs. incorrect transformer predictions on the test set.

7.4 Interpretability Findings

The Logistic Regression feature-importance analysis (Figure 9) and its corresponding exact SHAP explanation (Figure 10) are mutually consistent, as expected for a linear model with an exact explainer, and both align with domain intuition: the highest-magnitude positive-sentiment features are evaluative, appreciation-oriented terms, while the highest-magnitude negative-sentiment features are terms associated with disappointment and poor quality (see Figure 9 for the specific ranked terms). The LIME explanation (generated as an interactive HTML visualization within the companion notebook, highlighting per-word contribution to the LSTM model's prediction for a sample review) provides a complementary, model-agnostic local explanation and is available for inspection in Full_Pipeline_Advanced.ipynb.

The transformer's attention visualization (Figure 11) shows attention mass from the [CLS] token concentrated on a relatively small number of tokens rather than spread uniformly across the input, supporting the interpretation that the model has learned to identify specific sentiment-relevant spans rather than aggregating signal diffusely across the whole review – a desirable property for both performance and interpretability.

8 Limitations

The following limitations are stated proactively, as they reflect deliberate scope and resource trade-offs rather than oversights:

Subsampled dataset. A balanced subsample of the full 50,000-review IMDB corpus is used (Section 3) rather than the complete dataset, to keep total training time tractable across eleven models. Increasing the subsample size is the most direct lever for improving absolute performance across every model if additional compute/time is available.

Shared hyperparameter search for the RNN family. As discussed in Section 4.5, hyperparameters are tuned once using LSTM as a representative proxy and applied to all

four recurrent architectures, rather than tuning each independently.

Single transformer model. Only DistilBERT is fine-tuned in this study; a broader comparison would include BERT, RoBERTa, and/or ALBERT.

HMM formulation. A simple per-class generative HMM is used rather than more expressive variants (e.g., autoregressive HMMs or HMM-GMM hybrids), and, as discussed in Section 7.2, is arguably not the best-suited technique for this task architecturally – it is included because comparison against a mismatched baseline is itself informative.

Limited interpretability coverage. LIME is applied to one example on one recurrent model; SHAP is applied to one example batch on the Logistic Regression model only; attention visualization covers only the transformer's last layer, averaged across heads. A comprehensive explainability study would extend each method to every model and, for the transformer, every layer and head individually.

Small hyperparameter grids for tree ensembles. Random Forest and XGBoost hyperparameter grids are intentionally small (two values per parameter) to keep total notebook runtime reasonable.

Recurrent model training set size. As discussed in Section 7, the RNN family's underperformance relative to classical and HMM baselines is attributed to overfitting on a comparatively small training set (3,654 reviews) relative to the embedding layer's parameter count. This is a direct consequence of the dataset subsampling limitation above, rather than an independent limitation, but is called out separately because it materially shapes the model-comparison results and should be the first thing re-evaluated if this study is repeated at larger scale.

9 Conclusion and Future Work

This report presented a controlled, methodologically documented comparison of eleven sentiment classification models spanning classical machine learning, sequential/generative modeling, and transformer-based transfer learning, evaluated identically on the IMDB movie review dataset. The fine-tuned DistilBERT transformer achieved the best overall performance (F1 = 0.922, ROC-AUC = 0.969), substantially ahead of the next-best model, a linear SVM on TF-IDF features (F1 = 0.874). More notably, the comparison revealed that every recurrent neural network model in this study underperformed every classical machine learning model and the Hidden Markov Model, a result traced to overfitting on a comparatively small training set (3,654 reviews) relative to the RNN family's parameter count, rather than to any inherent weakness of recurrent architectures in general. The Simple RNN's near-chance performance (F1 = 0.563) offered a clean empirical confirmation of the vanishing-gradient motivation for gated architectures such as LSTM and GRU. Beyond raw performance comparison, this study emphasized methodological rigor: identical data splits across all models, hyperparameter tuning appropriate to each model's search-space size and training cost, explicit justification of preprocessing choices, and a direct, evidence-based discussion of architectural suitability – particularly for the Hidden Markov Model, whose fit to this task is critically examined rather than assumed.

Future work could extend this study along several axes: evaluating on the full 50,000-review IMDB corpus rather than a subsample; independently tuning each recurrent architecture rather than sharing a proxy search; fine-tuning and comparing additional transformer models (BERT, RoBERTa, ALBERT, DeBERTa-v3); extending interpretability analysis (SHAP and attention visualization)

to every model rather than representative examples; and packaging the best-performing model(s) behind a production inference API with appropriate monitoring, alongside the interactive Streamlit demonstration application developed alongside this report.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623–2631.

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(1), 281–305.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of EMNLP 2014*, 1724–1734.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.

Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.

Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics*, 142–150.

McCallum, A., & Nigam, K. (1998). A comparison of event models for naive Bayes text classification. *AAAI-98 Workshop on Learning for Text Categorization*, 41–48.

Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. *Proceedings of EMNLP 2002*, 79–86.

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257–286.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144.

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

A Appendix: Reproducibility

A.1 Code and Data Availability

The complete implementation – data pipeline, all eleven model implementations, hyperparameter optimization code, evaluation code, and the interactive Streamlit demonstration application – is provided alongside this report as a self-contained Jupyter notebook (Full Pipeline Advanced.ipynb) and companion project files.

A.2 Environment

All experiments use a fixed random seed (42) applied to Python's random module, NumPy, TensorFlow, and PyTorch. Package versions are pinned in requirements.txt to ensure exact reproducibility; see the project README for environment setup instructions.

A.3 Regenerating This Report's Results

All quantitative results, tables, and figures in this report were generated from a completed run of Full Pipeline Advanced.ipynb, which produces models/metrics.json (all quantitative results), models/config.json (final tuned hyperparameters), and the full set of figure files (confusion matrices, ROC curves, learning curves, Optuna trial plots, SHAP/attention visualizations) embedded throughout Sections 3–7. Re-running the notebook with a different random seed, larger N_SAMPLES, or modified hyperparameter search spaces will regenerate all of these artifacts, and this report can be updated accordingly by substituting the new figures and metric values.